武汉大学实验报告

院系：遥感信息工程学院　专业：遥感科学与技术

实验名称	基于Node.js和Socketio.js的聊天页面设计					指导教师	杨代琴
姓名	郭宇航	年级	2024级	学号	2024302131147	成绩	

一、实验目的

1）了解 JavaScript 编程语言，初步熟悉 Node.js 环境下的服务器开发，结合 Express 框架搭建 Web 服务器，完成聊天室的后端逻辑设计。

2）结合 Socket.io 基础教程，理解 WebSocket 协议在实时通信中的应用原理，实现客户端与服务器的双向实时数据传输。

3）理解实时通信中的数据同步机制，解决消息广播等关键问题。

二、实验的主要内容

1.环境配置:配置 Node.js 开发环境，安装 Express（Web 框架）、Socket.IO（实时通信库）等依赖包，完成项目基础文件的搭建（如index.js服务器端，index.html客户端页面）

2.服务器端设计

基于 Express 创建 Web 服务器，集成 Socket.IO 实现 WebSocket 通信。

处理用户连接、断开连接事件。开发消息处理机制：接收客户端消息并广播给所有在线用户，支持系统消息（如用户加入 / 离开通知）。

3.客户端设计

开发聊天室前端页面，主要包括消息展示区、输入框界面，实现用户之间通过消息交流，通过 Socket.IO 客户端库与服务器建立连接，处理消息收发、用户状态更新等事件。

4. 功能优化（教程之外）

支持用户自定义昵称（第一次进入页面时提示）和修改昵称

为用户自动生成头像和独特UID

显示用户ip地址

显示用户进入、离开聊天室以及发送消息的时间

提供用户列表（实时显示在线与离线用户）

切换用户列表（移动端适配）

支持响应式布局，在移动端通过切换按钮控制用户列表的显示 / 隐藏。

本地存储用户信息（昵称、设备 ID），避免重复输入。

三、实验方法

1.相关技术

项目管理:使用 package.json 管理项目依赖（如 express、socket.io、调用 ES 模板等）

后端：Node.js （开源、跨平台的JavaScript运行环境）+ Express（搭建 HTTP 服务器，Web应用开发框架）+ Socket.IO（处理实时通信）。

前端：HTML（页面输入框、各种按钮、聊天区域等） + CSS（聊天室页面布局与样式）+JavaScript（实现客户端与服务器端的交互）+ Socket.IO 客户端（与服务器通信）。

2.开发步骤

1）搭建服务器:使用 project.json 管理项目文件，编写 index.js，创建 Express 服务器并集成 Socket.IO，监听客户端连接事件，定义用户注册、消息广播等事件处理函数。

如 import express from 'express';//导入 express 框架

import { createServer } from 'node:http';//从 Node.js 的 http 模块导入创建 http 服务器方法

import { Server } from 'socket.io';//从 socket.io 导入服务器类，通过 socket.io 实现双向实时信息交流

const app = express();//创建 Express 应用实例 app（在本次实验中主要用于定义路由）

const server = createServer(app);//基于应用实例 app 创建 http 服务器实例 server

const io = new Server(server);//创建 Socket.IO 服务器实例 io 并附加到 server 上，实现在服务器 server 上同时处理客户端网页发送来的 http 请求并利用 WebSocket 实现实时通信

io.on('connection', (socket) => {监听用户连接事件，socket 为 html 中创建的 io 对象

```
// 监听用户注册事件
 socket.on('register user', (user) => {
     io.emit('update user list', updatedUserList); // 更新在线用户列表并广播
 });

 // 监听消息发送事件（来自 html<script>中发送）
 socket.on('chat message', (msg) => {
   io.emit('chat message', msg); // 广播消息给所有客户端
```

```
  });
});
```

2）设计客户端页面开发 index.html，实现消息展示区、输入框、用户列表等 UI 组件，通过 CSS 实现响应式布局（如移动端用户列表折叠）。
<style>设计内部 CSS 样式<style>
<body>展示用户可见内容（文字，按钮，图片等）<body>
<script>嵌入 JavaScript 代码，定义全局变量与函数，通过 socket.emit 向服务器端发送事件，服务器端通过 socket.on 监听<script>

3）实现前后端交互，在客户端通过 socket.io 连接服务器，发送 / 接收消息，处理用户列表更新。客户端加载 index.html 后，通过 <script src="/socket.io/socket.io.js"></script> 引入 Socket.io 客户端库，随后执行 const socket = io(); 向服务器发起连接请求。服务器端通过 io.on('connection', (socket) => { ... }) 监听客户端连接，当客户端连接成功时，服务器会为该客户端创建一个唯一的 socket 对象，用于后续单独通信。我通过 chat message 事件（用户发送信息）来分析连接过程中客户端与服务器端的交互流程
/html/

```
const socket = io();//客户端与 Socket.IO 服务器连接，
// 发送消息
form.addEventListener('submit', (e) => {
  e.preventDefault();
  socket.emit('chat message', { text: input.value, nickname: nickname });//向服务器发送 chat message 事件
  input.value = '';
});
// 接收消息并显示
socket.on('chat message', (msg) => {
  const li = document.createElement('li');
  li.textContent = `${msg.nickname}: ${msg.text}`;
  messages.appendChild(li);
});
```

/js/

```
socket.on('chat message', (data) => {
  const { text: msg, deviceId, nickname } = data;
```

```
 // 构建包含所有需要的字段的消息对象
 const messageTime = new Date().toLocaleString();
 const chatMessage = {
    type: 'chat',
    nickname: nickname,
    text: msg,
    deviceId: deviceId,
    ip: formattedIp,
    time: messageTime
  };
  console.log('message: ' + `${nickname} (${deviceId}) [${formattedIp}]:${msg}`);
  io.emit('chat message', chatMessage);
});
```

在这个事件中，
①用户在网站中的输入框输入信息并点击提交按钮后，被 form.addEventListener 函数监听到，调用 socket.emit 函数向服务器端发送 chat message 事件与消息数据
②服务端通过 socket.on 函数监听到消息后对消息进行处理，在通过 io.emit 函数广播到所有在线客户端
③各在线客户端通过 socket.on 函数监听到服务器发来的消息，通过 message.appendChild 函数将消息添加到 html<style>中预先准备好的容器中展示到网页上。
也就是说，连接过程中信息交互的流程为:客户端发送信息->服务器端接收并处理信息，广播给客户端->客户端接收信息并显示在网页上

四、实验成果



图为多台设备及同台不同浏览器访问网站进行交流的聊天记录。

与 Socket.io 官网教程相比，图中所增加的功能有:

1.用户在首次访问网站时可以设置一个昵称，且后续能够更改，更改后广播给所有在线用户

```
let nickname = localStorage.getItem('nickname');
    while (!nickname || nickname.trim() === '') {
      nickname = prompt('请输入您的昵称：').trim();
      if (!nickname) {
        alert('昵称不能为空，请重新输入！');
      }
    }
    localStorage.setItem('nickname', nickname); // 保存昵称到本地存储


    // 渲染右上角昵称显示
function renderNickname() {
  // 先移除已存在的显示元素（避免重复）
  const existing = document.getElementById('nicknameDisplay');
  if (existing) existing.remove();

// 创建新的容器元素，包含头像和昵称
  const container = document.createElement('div');
  container.id = 'nicknameDisplay';
  container.style.display = 'flex';
  container.style.alignItems = 'center';

 // 创建头像
  const avatar = document.createElement('div');
  const firstChar = getFirstChar(nickname);
  const avatarColor = getAvatarColor(deviceId);
  avatar.className = 'avatar';
  avatar.style.backgroundColor = avatarColor;
  avatar.textContent = firstChar;

  // 创建昵称文本
  const nicknameText = document.createElement('span');
  nicknameText.textContent = nickname;
  nicknameText.style.marginLeft = '8px'; // 头像和昵称之间的间距

  // 组装容器
  container.appendChild(avatar);
  container.appendChild(nicknameText);
  document.body.appendChild(container); // 添加到页面
}

  //更改昵称
    document.getElementById('changeNickname').addEventListener('click', () => {
  const newNickname = prompt('请输入新的昵称：', nickname).trim();
  if (newNickname && newNickname !== nickname) {
    const oldNickname = nickname; // 保存旧昵称
    nickname = newNickname;
    localStorage.setItem('nickname', nickname);
    alert('昵称已更新为：' + nickname);
     document.getElementById('currentNickname').textContent = `${nickname}`;
     renderNickname();
    // 向服务器发送昵称修改通知
    socket.emit('nickname changed', {
      oldNickname: oldNickname,
      newNickname: newNickname,
      deviceId: deviceId // 用于标识唯一用户
    });
  } else if (!newNickname) {
    alert('昵称不能为空！');
  }
});
```

2.根据用户昵称为用户生成一个头像

```
// 生成头像背景色
function getAvatarColor(str) {
  // 根据字符串生成哈希值
  let hash = 0;
  for (let i = 0; i < str.length; i++) {
    hash = str.charCodeAt(i) + ((hash << 5) - hash);
  }
  // 转换哈希值为RGB颜色
  const c = (hash & 0x00FFFFFF)
    .toString(16)
    .toUpperCase();
  // 补全6位
  return '#' + '00000'.substring(0, 6 - c.length) + c;
}
// 获取昵称首字符
function getFirstChar(nickname) {
  if (!nickname) return '';
  // 中文取第一个字符，英文取首字母大写
  return nickname.charAt(0).toUpperCase();
}
```

```
    // 普通消息样式（头像+昵称+内容）
    const avatarColor = getAvatarColor(msg.deviceId); // 基于deviceId生成固定颜色
const firstChar = getFirstChar(msg.nickname);
    item.innerHTML = `
     <div class="message-container">
       <div class="avatar" style="background-color: ${avatarColor}">${firstChar}</div>
       <span class="nickname">${msg.nickname}</span>
       <span class="message-text">(${msg.deviceId})[${msg.ip}]: ${msg.text}</span>
       <span class="message-time">${msg.time}</span>
     </div>
    `;
  }
```

3.为每个用户生成独特的 UID（重启服务器也不发生改变）

```
  // 生成或读取设备唯一标识（UUID）
let deviceId = localStorage.getItem('deviceId');
if (!deviceId) {
 const min = 10000000; // 最小8位数字
const max = 99999999; // 最大8位数字
const randomNum = Math.floor(Math.random() * (max - min + 1)) + min;
deviceId = randomNum.toString(); // 转换为字符串
  localStorage.setItem('deviceId', deviceId);
}
```

4.在每个用户发送的消息后面附上发送消息的时间，发送消息的前面附上头像、昵称、UID 以及 ip 地址（2.第二张代码截图）

```
socket.on('user connected', (data) => {
    const { nickname } = data;
    // 获取当前时间并格式化
    const joinTime = new Date().toLocaleTimeString(); // 格式: 2024/9/25 15:30:45
    // 构造系统消息（区分普通消息，方便前端样式处理）
    const systemMsg = {
      type: 'system', // 标识为系统消息
      content: `${nickname}[${formattedIp}]进入了聊天室`,
      time: joinTime
    };
    // 广播给所有用户（包括自己）
    io.emit('chat message', systemMsg);
  });
   socket.on('user connected', (user) => {
    socket.nickname = user.nickname; // 保存用户昵称到socket对象
    // 其他连接逻辑...
  });
```

5.在每个用户进入与离开聊天室时广播给所有在线用户，同时在命令行窗口显示连接/断开连接，添加了用户列表，实时更新在线与离线用户（服务器重启后重置）

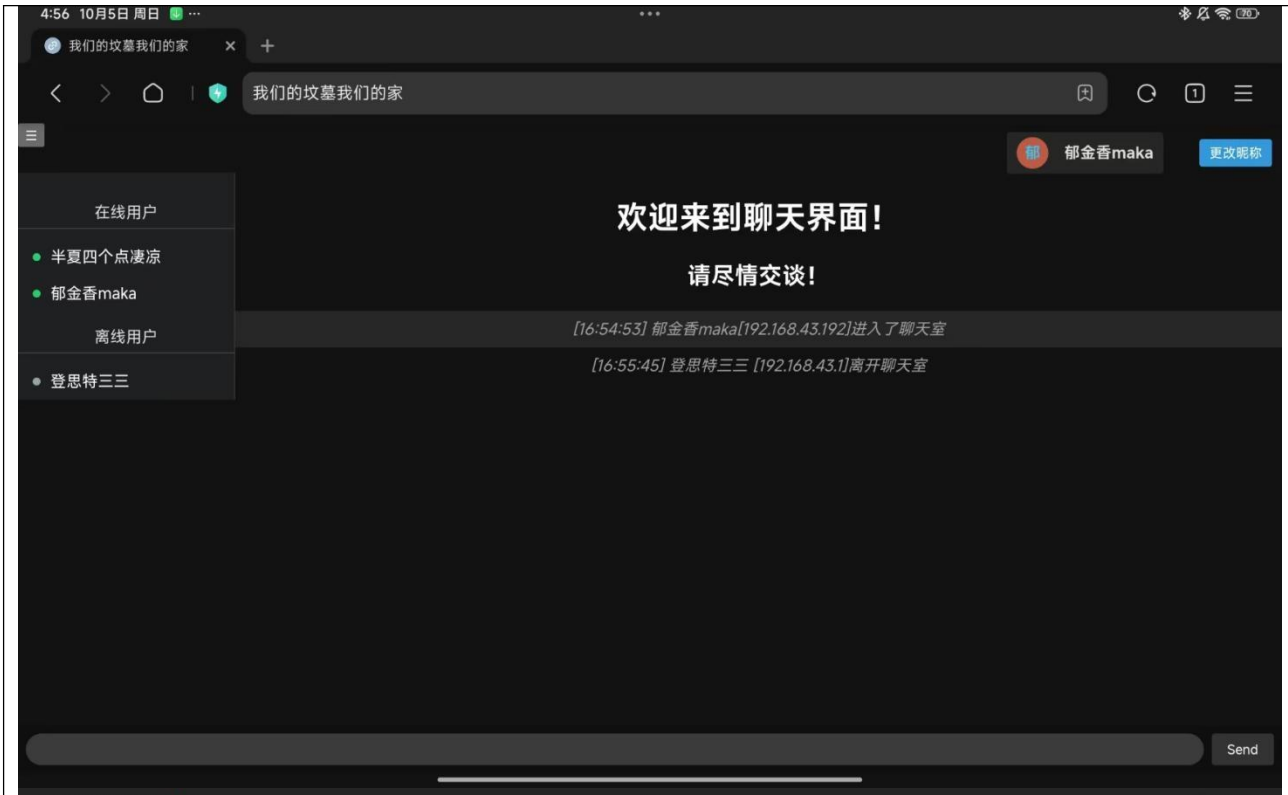
```
// 监听断开连接事件
socket.on('disconnect', () => {
  if (socket.nickname) {
    // 在命令行显示用户断开连接
    console.log(` ${socket.nickname}[${formattedIp}] 已断开连接`);

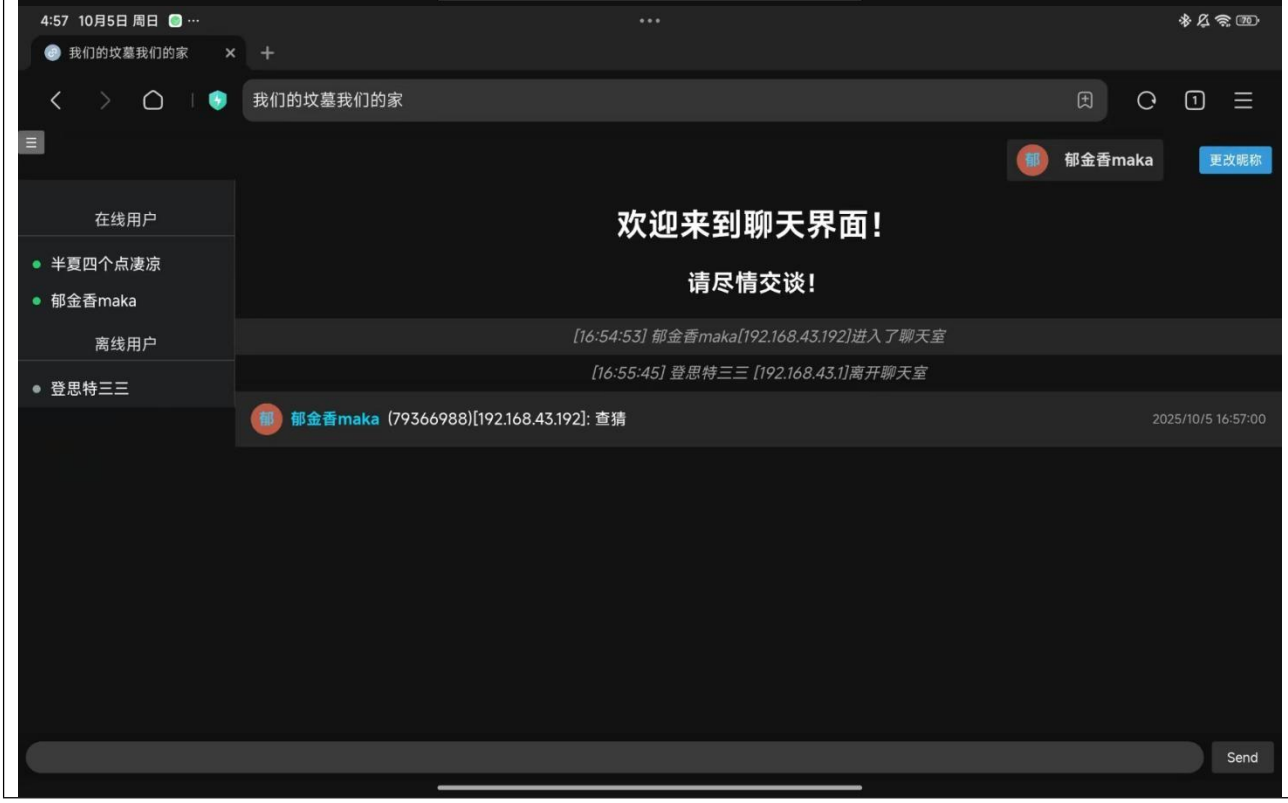
    // 向所有客户端广播用户离开消息
    io.emit('chat message', {
      type: 'system',
      content: `${socket.nickname} [${formattedIp}]离开聊天室`,
      time: new Date().toLocaleTimeString() // 添加时间戳
    });
  }
  // 查找断开连接的用户
  const offlineUser = users.online.find(u => u.socketId === socket.id);
  if (offlineUser) {
    // 从在线列表移除并添加到离线列表
    users.online = users.online.filter(u => u.socketId !== socket.id);
    users.offline.push(offlineUser);
    // 广播更新用户列表
    io.emit('update user list', users);
  }
});
```

剩下几张为不同设备的网站截图。



6.在手机端增添一个可使用的按钮，用于显示与隐藏用户列表


4:56 10月5日 周日
我们的坟墓我们的家
我们的坟墓我们的家
郁金香maka
更改昵称
在线用户
半夏四个点凄凉
郁金香maka
离线用户
登思特三三
欢迎来到聊天界面!
请尽情交谈!
[16:54:53] 郁金香maka[192.168.43.192]进入了聊天室
[16:55:45] 登思特三三 [192.168.43.1]离开聊天室
Send


4:57 10月5日 周日
我们的坟墓我们的家
我们的坟墓我们的家
郁金香maka
更改昵称
在线用户
半夏四个点凄凉
郁金香maka
离线用户
登思特三三
欢迎来到聊天界面!
请尽情交谈!
[16:54:53] 郁金香maka[192.168.43.192]进入了聊天室
[16:55:45] 登思特三三 [192.168.43.1]离开聊天室
郁金香maka (79366988)[192.168.43.192]: 查猜
2025/10/5 16:57:00
Send



五、实验结论与总结

比那什么MFC好玩多了！（虽然除了教程以外增加的功能基本都是交给AI来写然后我自己再稍微调一点参数就是了）

遇到的第一个问题是本机ip地址的返回值，不显示真实ip地址而是IPv4 回环地址127.0.0.1，

最终也是在AI的帮助下成功解决。
还有一些问题是在不断增加功能中产生的，比如我一开始没有意识到html中socket.on('chat message', (msg) => {})的msg与js中msg是对应的，在增添更改代码之后出现无法获取文本的情况，后面我重新理解这几个函数的作用原理之后改过来了。
因为我本身还没有系统的学过JavaScript这门编程语言，所以很多代码都得直接交给AI来生成，但通过反复询问AI和调整参数，我也是对这些函数的原理有一定的理解了，上面生成UID的函数和获取昵称的函数是我在AI的帮助下自己扣出来的（虽说这两个比较简单就是了），以及实验方法-开发步骤中对于各句代码
功能的理解也都是基于AI的回答上的自己的认识，可能不太精准。不过在反复拷打AI和思考AI给出的代码的过程中，我也是体验到了实时通信应用的开发流程，深刻理解了 WebSocket 技术的优势，对Socket.io基于JavaScript实现的服务器端与客户端交互实时通信的功能有了很强的认识。

教师评语

指导教师_____ 2025年_____月_____日